PARTIE 3 | Mémo Bash pour coder des scripts (avec explications)

1) Shebang (ligne 1 obligatoire)

Le *shebang* indique **avec quel interpréteur** exécuter le fichier.

```
#!/usr/bin/env bash
# ↑ Demande à l'environnement de trouver 'bash' (portable entre systèmes).
#   Toujours en 1re ligne, sans espace avant.
```

---

2) Créer, rendre exécutable, exécuter

```
nano monscript.sh       # Ouvre/ crée le fichier dans l'éditeur nano.
chmod +x monscript.sh   # Ajoute le droit d'exécution au fichier.
./monscript.sh          # Exécute le script du dossier courant.
# (évite 'sh monscript.sh' qui peut appeler un shell POSIX différent)
```

À retenir : le `./` est nécessaire car le dossier courant n'est pas dans `$PATH`.

---

3) Affichage et commentaires

```
# Tout ce qui est après # est ignoré par l'interpréteur.
echo "Bonjour"          # Affiche du texte et ajoute une nouvelle ligne.
printf "Salut %s\n" Bob # printf = formatage précis, pas de retour à la ligne
implicite.
```

---

4) Variables, guillemets, substitutions

Affectation + expansion

```
NOM="Alice"             # Pas d'espaces autour de '='
echo "Bonjour $NOM"     # $NOM est remplacé par sa valeur (expansion).
```

Guilles doubles vs simples

```
MSG="jour"
echo "Bon $MSG"         # → Bon jour   (expansion activée)
echo 'Bon $MSG'         # → Bon $MSG   (pas d'expansion en quotes simples)
```

Substitution de commande

```
DATE="$(date +%F)"      # Exécute 'date +%F' et récupère son résultat.
echo "Nous sommes le $DATE"
```

Petite arithmétique

```
NB=$(( 3 + 2 ))         # $(( ... )) évalue une expression arithmétique.
echo "Résultat: $NB"
```

Piège classique : toujours **citer** les variables contenant des chemins/chaînes :

```
fichier="mon fichier.txt"
# Sans guillemets, l'espace coupe en 2 arguments → bug
cat "$fichier"          # Toujours "$var"
```

---

5) Arguments du script

Les valeurs passées après le nom du script.

```bash
#!/usr/bin/env bash
echo "Nom du script : $0"                    # $0 = chemin/nom du script
echo "Premier argument : ${1:-aucun}"        # $1 = 1er arg ; :- <- valeur par défaut
si vide
echo "Tous les arguments : $@"               # $@ = liste des arguments
```

Astuce : `${var:-def}` évite les “variable: unbound” quand elle n’est pas fournie.

---

6) Codes de retour (succès/échec)

Chaque commande renvoie un code : `0 = succès`, `≠0 = échec`.

```bash
ls >/dev/null
echo "Code précédent : $?"      # Affiche 0 si tout s'est bien passé.

ls /chemin/qui/nexiste/pas 2>/dev/null
echo "Code précédent : $?"      # ≠ 0 → il y a eu une erreur.
```

Pourquoi c’est vital (DevOps) : tu pourras enchaîner des actions **si et seulement si** la précédente a réussi.

---

7) Tests et conditions

Utilise `[[ ... ]]` (Bash) : plus sûr que `[ ... ]` (POSIX).

Tests de fichiers

```
FILE="demo.txt"

if [[ -e "$FILE" ]]; then        # -e : existe (fichier ou dossier)
  echo "Existe"
fi

if [[ -f "$FILE" ]]; then        # -f : fichier régulier
  echo "C'est un fichier"
fi

if [[ -d "$FILE" ]]; then        # -d : dossier
  echo "C'est un dossier"
else
  echo "Ce n'est pas un dossier"
fi

if [[ -r "$FILE" && -w "$FILE" ]]; then  # -r/-w : lisible/écrivable
  echo "Lisible et modifiable"
fi
```

Comparateurs numériques (dans `[[ ]]`)

Op	Signification	Exemple
-eq	égal	`[[ $a -eq $b ]]`
-ne	différent	`[[ $a -ne $b ]]`
-lt	inférieur	`[[ $a -lt $b ]]`
-le	inférieur ou égal	`[[ $a -le $b ]]`
-gt	supérieur	`[[ $a -gt $b ]]`
-ge	supérieur ou égal	`[[ $a -ge $b ]]`

```
read -rp "Âge ? " age
if [[ $age -ge 18 ]]; then      # -ge : >=
  echo "Majeur"
else
  echo "Mineur"
fi
```

Comparateurs de chaînes

Op	Signification	Exemple
==	égal	`[[ $s == "ok" ]]`
!=	différent	`[[ $s != "ok" ]]`
-z	chaîne vide	`[[ -z "$s" ]]`
-n	chaîne non vide	`[[ -n "$s" ]]`

```
user=""
if [[ -z "$user" ]]; then
  echo "Rien saisi"
fi
```

Condition complète

```
if   [[ condition1 ]]; then
  # commandes si vrai
elif [[ condition2 ]]; then
  # autre branche
else
  # par défaut
fi
```

---

8) Boucles

Parcourir une séquence

```
for i in {1..3}; do
  echo "Itération $i"
done
```

Parcourir des fichiers (globbing)

```
for f in *.txt; do
  echo "Fichier: $f"
done
```

Piège : si rien ne matche, `*.txt` peut rester littéral selon options → protège avec tests :

```
shopt -s nullglob       # (option) *.txt -> liste vide si aucun match
for f in *.txt; do
  [[ -e "$f" ]] || continue
  echo "Fichier: $f"
done
```

---

9) Lecture utilisateur

```
read -rp "Votre nom ? " nom   # -r : pas d'interprétation de backslashes ; -p :
prompt
echo "Bonjour, $nom"
```

---

10) Fonctions

Factorise du code réutilisable.

```
bonjour() {
  # $1 = 1er argument de la fonction
  echo "Bonjour, ${1:-anonyme}"
}

bonjour "Alice"           # Appel
bonjour                   # → 'anonyme' (valeur par défaut)
```

Fonction qui “échoue” volontairement (utile en DevOps) :

```
die() {
  echo "ERREUR: $*" >&2    # >&2 = sur la sortie d'erreur
  exit 1                   # code ≠ 0 → échec
}
# die "Fichier introuvable"
```

---

11) Menu interactif (exemple minimal, commenté)

```bash
#!/usr/bin/env bash
# Mini-menu interactif : informations système ou sortie.

while true; do
  echo "1) Info noyau (uname -a)"
  echo "2) Date/Heure"
  echo "3) Quitter"
  read -rp "Votre choix : " choix

  case "$choix" in
    1) uname -a ;;                      # Affiche infos noyau
    2) date "+%F %T" ;;                 # Affiche date & heure
    3) echo "Au revoir"; exit 0 ;;      # Sortie propre
    *) echo "Choix invalide" ;;         # Cas par défaut
  esac
done
```

Points clés :

- `while true` maintient le menu jusqu'à `exit`.
- `case ... esac` rend la logique de choix **lisible**.
- Un `exit 0` indique une sortie **normale**.

---

12) Bonnes pratiques minimales (pour des scripts qui tiennent)

```bash
#!/usr/bin/env bash
set -euo pipefail
# -e : stoppe dès qu'une commande échoue
# -u : interdit l'usage de variables non définies
# -o pipefail : un pipeline échoue si un maillon échoue
IFS=$'\n\t'  # évite des surprises sur les espaces dans for/while
```

- **Toujours** mettre des guillemets autour des variables : `"$var"`.
- Préfère `[[ ... ]]` à `[ ... ]`.
- Préfère les *globs* (`for f in *.log`) à `for f in $(ls *.log)`.
- Écris des **messages clairs** quand tu échoues (et sors avec `exit 1`).

---

13) Script d'exemple "complet" (ultra-court, prêt à adapter)

```bash
#!/usr/bin/env bash
set -euo pipefail

usage() {
  echo "Usage: $0 <fichier>" >&2
  exit 1
}

file="${1:-}"; [[ -n "$file" ]] || usage
[[ -f "$file" && -r "$file" ]] || { echo "Fichier invalide: $file" >&2; exit 1; }

echo "Lignes (wc -l) et dernière ligne (tail -n 1) :"
wc -l "$file"
tail -n 1 "$file"
```

Ce qu'il montre : shebang, options de sûreté, fonction d'usage, vérifs d'arguments et de fichiers, messages d'erreur sur `stderr`, et 2 commandes utiles.

TL;DR (mini-fiche à mémoriser)

- Shebang : `#!/usr/bin/env bash`
- Variables : `VAR="val"` (jamais d'espace), toujours `"$VAR"`
- Tests : `[[ -f "$f" ]]`, numériques `-eq -ne -lt -le -gt -ge`, chaînes `-z -n == !=`
- Conditions : `if/elif/else/fi`
- Boucles : `for/while`
- Fonctions : `nom() { ... }`
- Menu : `case`
- Codes de retour : `0 ok`, `≠0 erreur`